Department of Artificial Intelligence (AI) & Data Science

Name:- Aaryan Nandkishor Raorane	Roll no:- S004
SAP ID:- 60018250070	Topic:- Tree Data Structure
Branch:- Artificial Intelligence & Data Science	

Experiment No. 7

Aim

To implement a Binary Search Tree using dynamic memory allocation and perform create, delete, search, and display operations through inorder, preorder, and postorder traversals.

Theory

Definition

A Binary Search Tree is a binary tree in which every value in the left subtree is smaller than the node value and every value in the right subtree is greater. Each node stores one data value and two links named left and right. Duplicate values are not inserted in this program.

Node Structure

```
struct Node
{
    int data;
    struct Node *left;
    struct Node *right;
};
```

Operations

- **Create and insert.** A node is allocated with malloc and the value moves left or right from the root until a NULL position is reached.
- **Delete.** A leaf is removed directly, a node with one child is replaced by that child, and a node with two children is replaced by its inorder successor.
- **Search.** The value is compared with the current node and only one subtree is followed each time.
- **Display.** Inorder prints left, root, right. Preorder prints root, left, right. Postorder prints left, right, root.

Pseudocode

Create node

```
BEGIN createNode(value)
    newNode = ALLOCATE NODE()
    IF newNode == NULL THEN
        PRINT "Memory allocation failed."
        RETURN NULL
    END IF
```

```
    newNode.data = value
    newNode.left = NULL
    newNode.right = NULL
    RETURN newNode
END createNode
```

Insertion

```
BEGIN insertNode(node, value)
    IF node == NULL THEN
        RETURN createNode(value)
    END IF
    IF value < node.data THEN
        node.left = insertNode(node.left, value)
    ELSE IF value > node.data THEN
        node.right = insertNode(node.right, value)
    END IF
    RETURN node
END insertNode
```

Minimum node

```
BEGIN findMin(node)
    temp = node
    WHILE temp.left != NULL DO
        temp = temp.left
    END WHILE
    RETURN temp
END findMin
```

Deletion

```
BEGIN deleteNode(node, value)
    IF node == NULL THEN
        RETURN NULL
    END IF
    IF value < node.data THEN
        node.left = deleteNode(node.left, value)
    ELSE IF value > node.data THEN
        node.right = deleteNode(node.right, value)
    ELSE
        IF node.left == NULL AND node.right == NULL THEN
            FREE(node)
            RETURN NULL
        ELSE IF node.left == NULL THEN
            temp = node.right
            FREE(node)
            RETURN temp
        ELSE IF node.right == NULL THEN
            temp = node.left
            FREE(node)
            RETURN temp
        ELSE
```

```
            temp = findMin(node.right)
            node.data = temp.data
            node.right = deleteNode(node.right, temp.data)
        END IF
    END IF
    RETURN node
END deleteNode
```

Search

```
BEGIN searchNode(node, value)
    IF node == NULL OR node.data == value THEN
        RETURN node
    END IF
    IF value < node.data THEN
        RETURN searchNode(node.left, value)
    END IF
    RETURN searchNode(node.right, value)
END searchNode
```

Inorder traversal

```
BEGIN inorder(node)
    IF node != NULL THEN
        inorder(node.left)
        PRINT node.data
        inorder(node.right)
    END IF
END inorder
```

Preorder traversal

```
BEGIN preorder(node)
    IF node != NULL THEN
        PRINT node.data
        preorder(node.left)
        preorder(node.right)
    END IF
END preorder
```

Postorder traversal

```
BEGIN postorder(node)
    IF node != NULL THEN
        postorder(node.left)
        postorder(node.right)
        PRINT node.data
    END IF
END postorder
```

Code

Program to create a BST and perform insertion, deletion, search, and three display traversals.

```
#include<stdio.h>
#include<conio.h>
```

```
#include<stdlib.h>

struct Node
{
	int data;
	struct Node *left;
	struct Node *right;
};

struct Node *root = NULL;

struct Node *createNode(int value)
{
	struct Node *newNode;
	newNode = (struct Node*)malloc(sizeof(struct Node));

	if(newNode == NULL)
	{
		printf("Memory allocation failed.\n");
		return NULL;
	}

	newNode->data = value;
	newNode->left = NULL;
	newNode->right = NULL;
	return newNode;
}

struct Node *insertNode(struct Node *node, int value)
{
	if(node == NULL)
	{
		return createNode(value);
	}

	if(value < node->data)
	{
		node->left = insertNode(node->left, value);
	}
	else if(value > node->data)
	{
		node->right = insertNode(node->right, value);
	}

	return node;
}

struct Node *findMin(struct Node *node)
{
```

```
	struct Node *temp;
	temp = node;

	while(temp->left != NULL)
	{
		temp = temp->left;
	}

	return temp;
}

struct Node *deleteNode(struct Node *node, int value)
{
	struct Node *temp;

	if(node == NULL)
	{
		return NULL;
	}

	if(value < node->data)
	{
		node->left = deleteNode(node->left, value);
	}
	else if(value > node->data)
	{
		node->right = deleteNode(node->right, value);
	}
	else
	{
		if(node->left == NULL && node->right == NULL)
		{
			free(node);
			return NULL;
		}
		else if(node->left == NULL)
		{
			temp = node->right;
			free(node);
			return temp;
		}
		else if(node->right == NULL)
		{
			temp = node->left;
			free(node);
			return temp;
		}
		else
		{
```

```
                temp = findMin(node->right);
                node->data = temp->data;
                node->right = deleteNode(node->right, temp->data);
            }
        }

        return node;
}

struct Node *searchNode(struct Node *node, int value)
{
        if(node == NULL || node->data == value)
        {
            return node;
        }

        if(value < node->data)
        {
            return searchNode(node->left, value);
        }

        return searchNode(node->right, value);
}

void inorder(struct Node *node)
{
        if(node != NULL)
        {
            inorder(node->left);
            printf("%d ", node->data);
            inorder(node->right);
        }
}

void preorder(struct Node *node)
{
        if(node != NULL)
        {
            printf("%d ", node->data);
            preorder(node->left);
            preorder(node->right);
        }
}

void postorder(struct Node *node)
{
        if(node != NULL)
        {
            postorder(node->left);
```

```
            postorder(node->right);
            printf("%d ", node->data);
        }
}

void main()
{
        int choice, value;

        clrscr();

        do
        {
            printf("\n1.Insert 2.Delete 3.Inorder 4.Preorder 5.Postorder 6.Search 7.Exit\n");
            printf("Enter choice: ");
            scanf("%d", &choice);

            switch(choice)
            {
                case 1:
                    printf("Enter value: ");
                    scanf("%d", &value);
                    if(searchNode(root, value) != NULL)
                    {
                        printf("Duplicate value is not inserted.\n");
                    }
                    else
                    {
                        root = insertNode(root, value);
                        printf("Value inserted.\n");
                    }
                    break;

                case 2:
                    printf("Enter value to delete: ");
                    scanf("%d", &value);
                    if(searchNode(root, value) == NULL)
                    {
                        printf("Value not found.\n");
                    }
                    else
                    {
                        root = deleteNode(root, value);
                        printf("Value deleted.\n");
                    }
                    break;

                case 3:
                    printf("Inorder: ");
```

```
                if(root == NULL)
                    printf("Tree is empty.");
                inorder(root);
                printf("\n");
                break;

            case 4:
                printf("Preorder: ");
                if(root == NULL)
                    printf("Tree is empty.");
                preorder(root);
                printf("\n");
                break;

            case 5:
                printf("Postorder: ");
                if(root == NULL)
                    printf("Tree is empty.");
                postorder(root);
                printf("\n");
                break;

            case 6:
                printf("Enter value to search: ");
                scanf("%d", &value);
                if(searchNode(root, value) == NULL)
                    printf("Value not found.\n");
                else
                    printf("Value found in BST.\n");
                break;

            case 7:
                printf("Exiting program.\n");
                break;

            default:
                printf("Invalid choice.\n");
        }
    } while(choice != 7);

    getch();
}
```

Output

```
1.Insert 2.Delete 3.Inorder 4.Preorder 5.Postorder 6.Search 7.Exit
Enter choice: 1
Enter value: 40
Value inserted.
```

```
1.Insert 2.Delete 3.Inorder 4.Preorder 5.Postorder 6.Search 7.Exit
Enter choice: 1
Enter value: 20
Value inserted.

1.Insert 2.Delete 3.Inorder 4.Preorder 5.Postorder 6.Search 7.Exit
Enter choice: 1
Enter value: 60
Value inserted.

1.Insert 2.Delete 3.Inorder 4.Preorder 5.Postorder 6.Search 7.Exit
Enter choice: 1
Enter value: 10
Value inserted.

1.Insert 2.Delete 3.Inorder 4.Preorder 5.Postorder 6.Search 7.Exit
Enter choice: 1
Enter value: 30
Value inserted.

1.Insert 2.Delete 3.Inorder 4.Preorder 5.Postorder 6.Search 7.Exit
Enter choice: 1
Enter value: 50
Value inserted.

1.Insert 2.Delete 3.Inorder 4.Preorder 5.Postorder 6.Search 7.Exit
Enter choice: 1
Enter value: 70
Value inserted.

1.Insert 2.Delete 3.Inorder 4.Preorder 5.Postorder 6.Search 7.Exit
Enter choice: 3
Inorder: 10 20 30 40 50 60 70

1.Insert 2.Delete 3.Inorder 4.Preorder 5.Postorder 6.Search 7.Exit
Enter choice: 4
Preorder: 40 20 10 30 60 50 70

1.Insert 2.Delete 3.Inorder 4.Preorder 5.Postorder 6.Search 7.Exit
Enter choice: 5
Postorder: 10 30 20 50 70 60 40

1.Insert 2.Delete 3.Inorder 4.Preorder 5.Postorder 6.Search 7.Exit
Enter choice: 6
Enter value to search: 30
Value found in BST.

1.Insert 2.Delete 3.Inorder 4.Preorder 5.Postorder 6.Search 7.Exit
```

```
Enter choice: 1
Enter value: 30
Duplicate value is not inserted.

1.Insert 2.Delete 3.Inorder 4.Preorder 5.Postorder 6.Search 7.Exit
Enter choice: 2
Enter value to delete: 20
Value deleted.

1.Insert 2.Delete 3.Inorder 4.Preorder 5.Postorder 6.Search 7.Exit
Enter choice: 3
Inorder: 10 30 40 50 60 70

1.Insert 2.Delete 3.Inorder 4.Preorder 5.Postorder 6.Search 7.Exit
Enter choice: 2
Enter value to delete: 99
Value not found.

1.Insert 2.Delete 3.Inorder 4.Preorder 5.Postorder 6.Search 7.Exit
Enter choice: 6
Enter value to search: 25
Value not found.

1.Insert 2.Delete 3.Inorder 4.Preorder 5.Postorder 6.Search 7.Exit
Enter choice: 7
Exiting program.
```

Applications

- Symbol tables in compilers store identifiers in an ordered tree.
- Database and file indexes use balanced tree variants to find records quickly.
- File systems represent folders and files as a tree.
- Expression trees evaluate operators and operands in calculators.
- Decision trees organise choices and outcomes for classification.

Conclusion

The Binary Search Tree was implemented using dynamic memory allocation. Nodes were created, inserted, deleted and searched, and the tree was displayed using inorder, preorder and postorder traversals. The inorder output was in ascending order and deletion preserved the BST property.